

Chapter 5: Real-world interfacing

5.1 Introduction

5.2 ADC/DAC

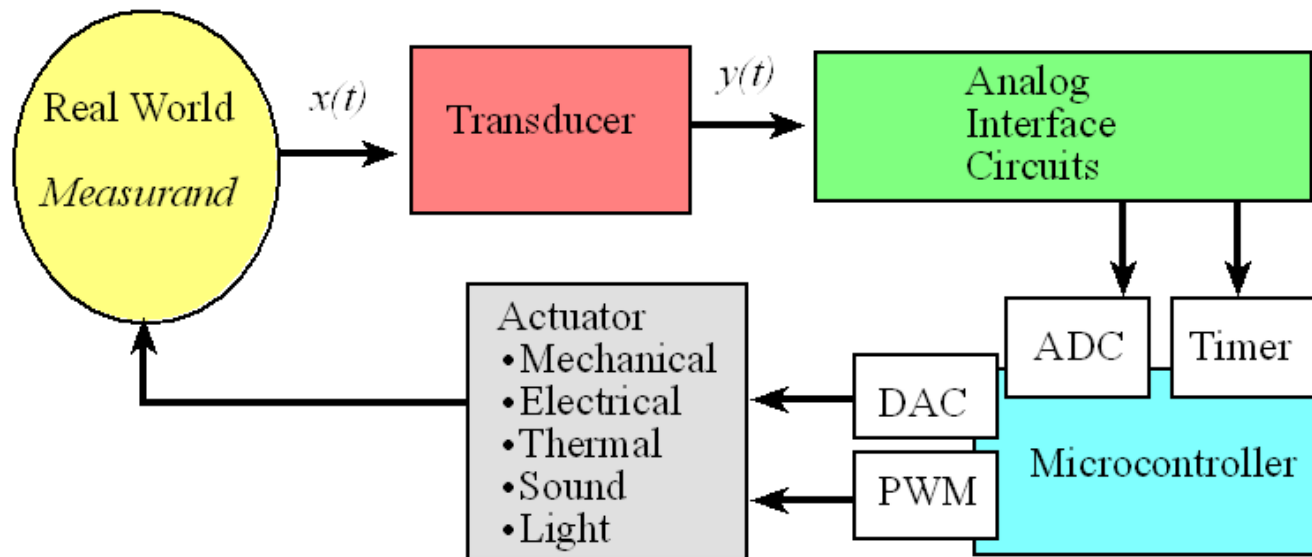
5.3 Interfacing with ADC

5.4 Dealing with noise and high power

5.5 Efficient I/O with interrupt and DMA

Introduction

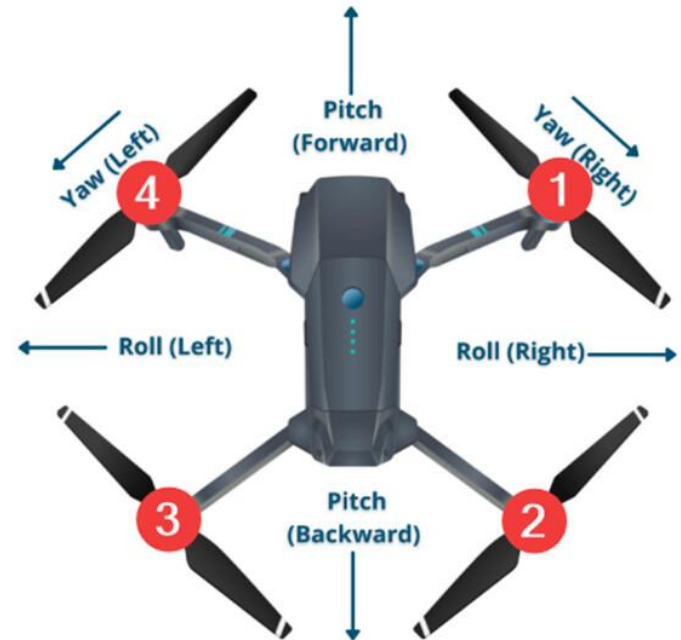
- ❑ Embedded system is an important part of digital measurement and control systems, e.g. drones, air conditional, ABS, ESP in cars,...
- | Input: real time data (temperature, humidity, orientation, speed...) depending on the physical objects/environment.
- | Output: actuator to control the objects.



©Jonathan Valvano and Ramesh Yerraballi

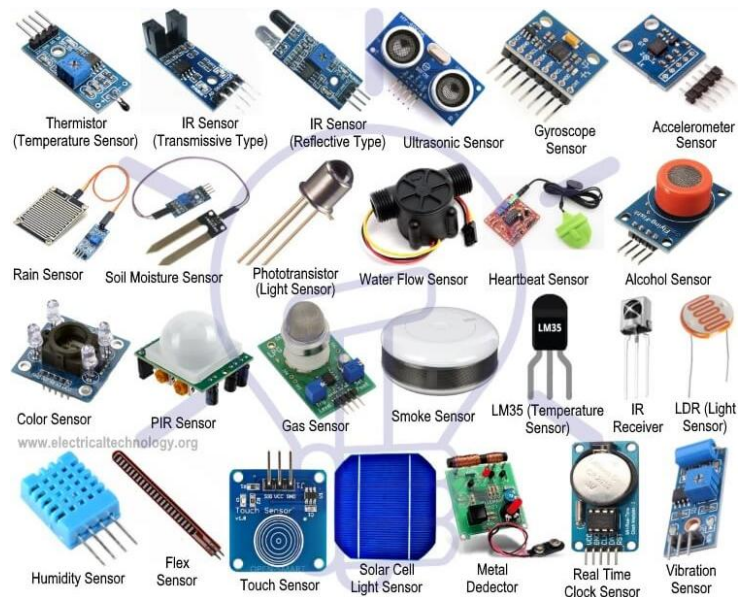
Introduction

- ❑ Example: flight control system on drones
- ❑ Input:
 - | Sensors: gyro, accelerometer, magnetometer, barometer, GPS...
 - | Command: throttle, velocity...
 - | System status: battery voltage, temperature,...
- ❑ Output
 - | ESC command for each motor
 - | Command for onboard camera



Sensors

- ❑ Devices to detect physical parameters and convert into electrical signal that can be measured.



Actuators

- ❑ Devices to convert energy into motion, to control a mechanism or system.
- ❑ Devices to receive command from control system to act upon the physical environment.
- ❑ Usually require large power compare to the embedded/control system.
- ❑ Example:
 - | Hydraulic actuators
 - | Pneumatic actuators
 - | Electrical actuators

ADC/DAC

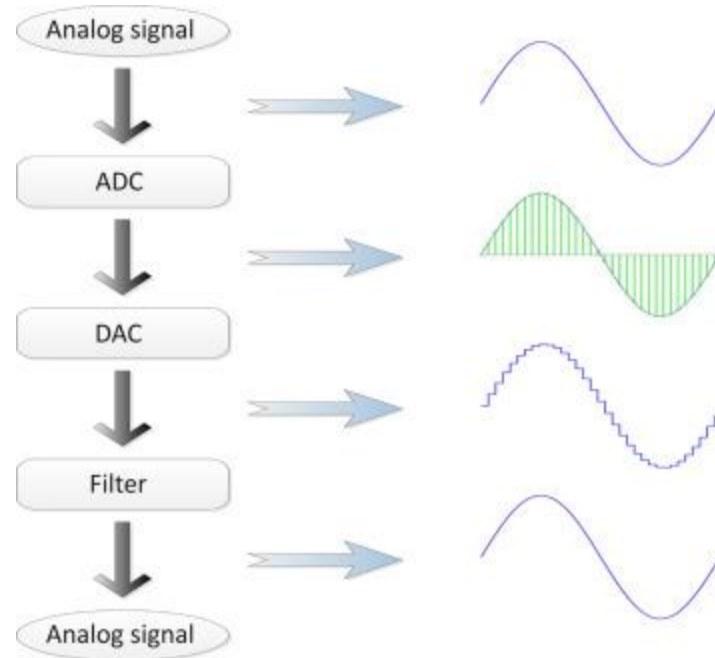
❑ ADC: Analog to Digital Converter

- | Electrical component to convert analog signals to digital signal, through sampling and quantization.

❑ DAC:

- | Electrical component to convert digital signals to analog signals.

$$U(t) \leftrightarrow X(k)$$



ADC/DAC

- ❑ Formula for ADC/DAC conversion

$$U = U_{ref} * \frac{x_{n-1}x_{n-2} \dots x_1x_0}{2^n}$$

U_{ref} : reference voltage, a parameter of the ADC/DAC

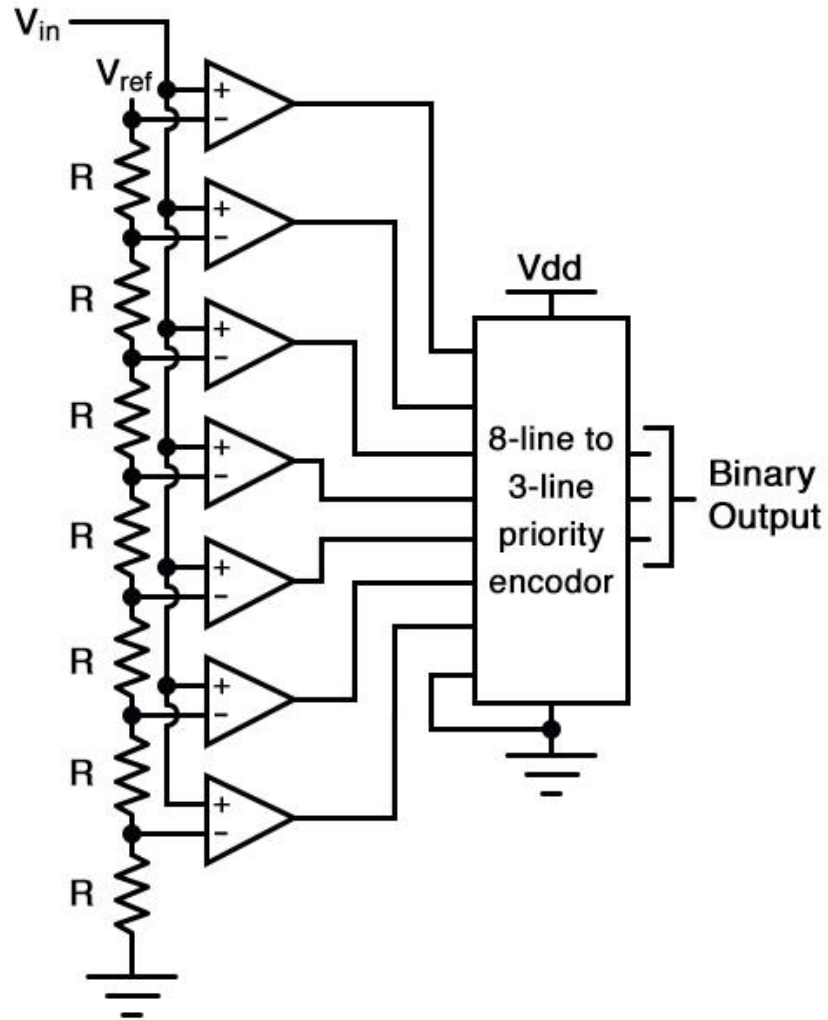
- ❑ ADC: given U_{in} , find corresponding X.
- ❑ DAC: given X, generate U_{out} .

❑ R-2R ladder DAC



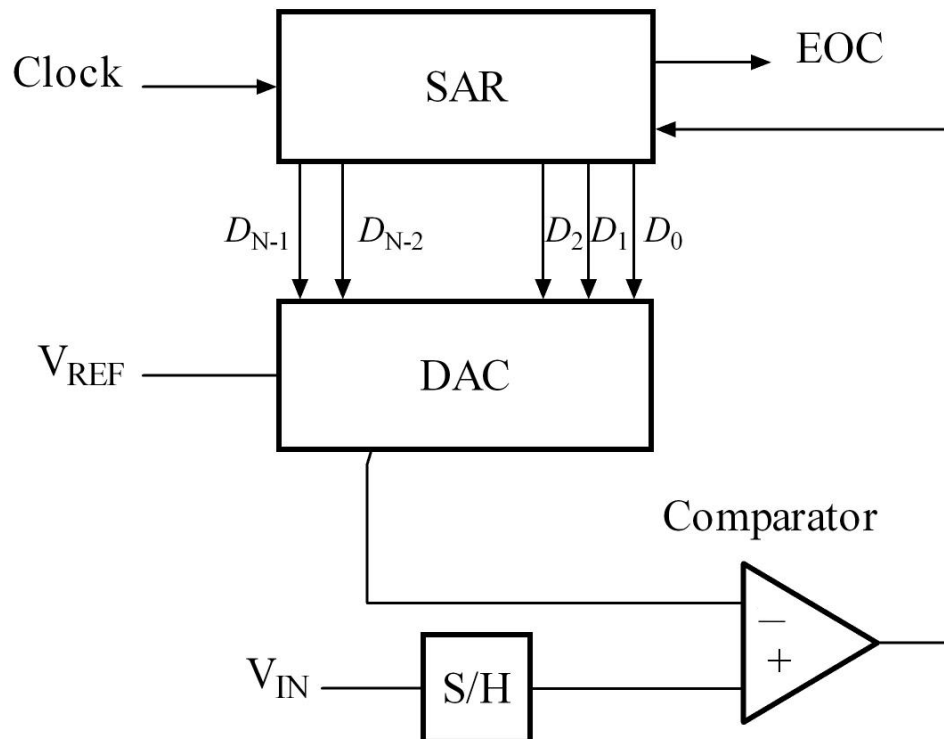
ADC/DAC

Flash ADC

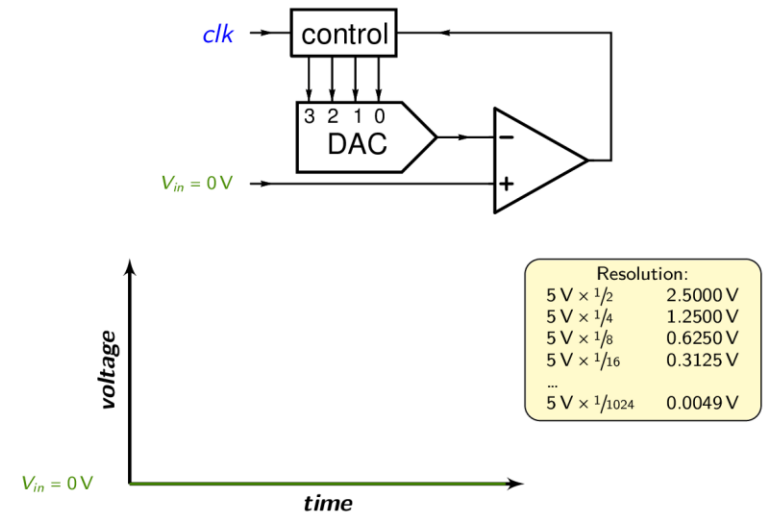


ADC/DAC

ADC successive approximation

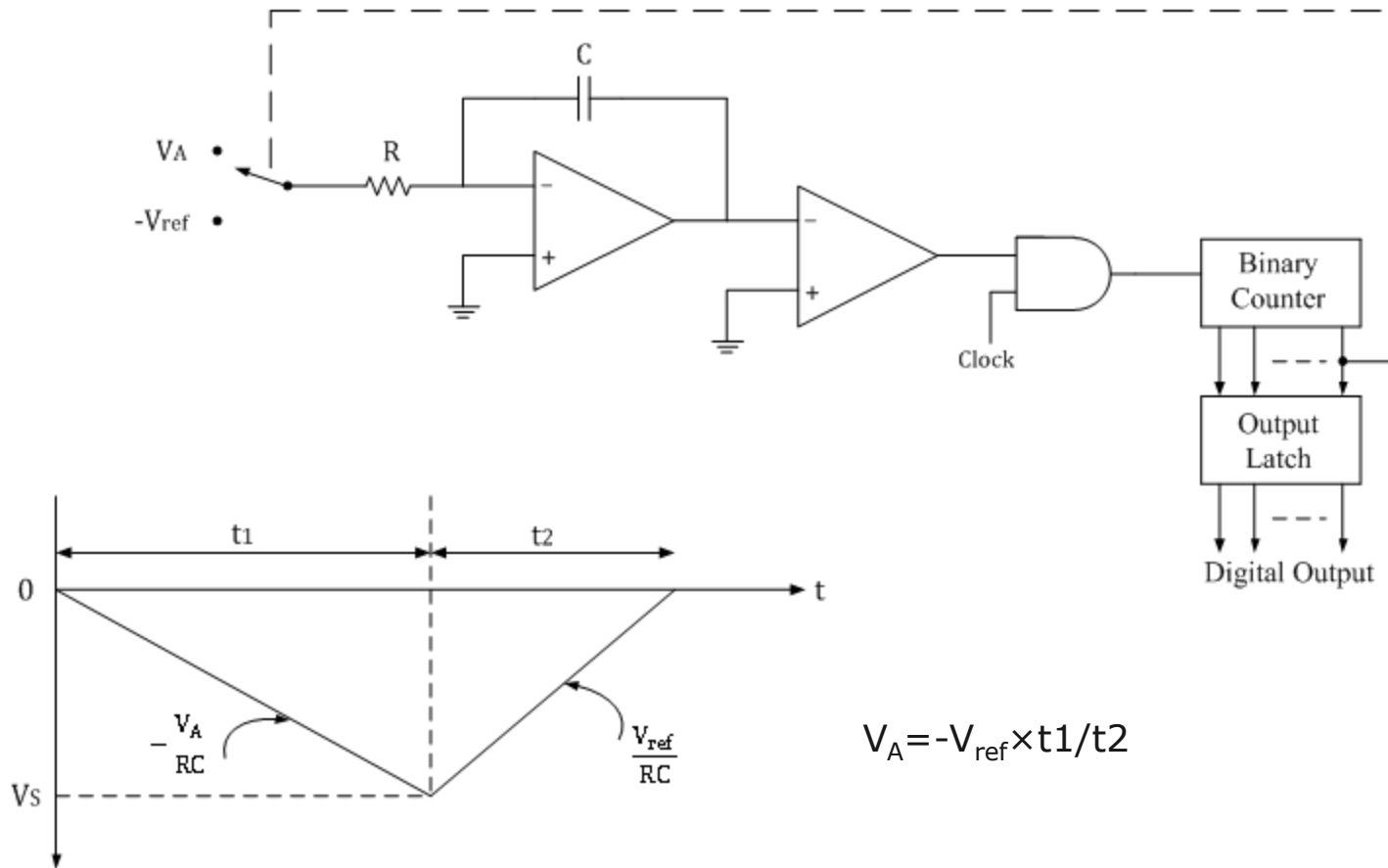


Successive Approximation – example of a 4-bit ADC



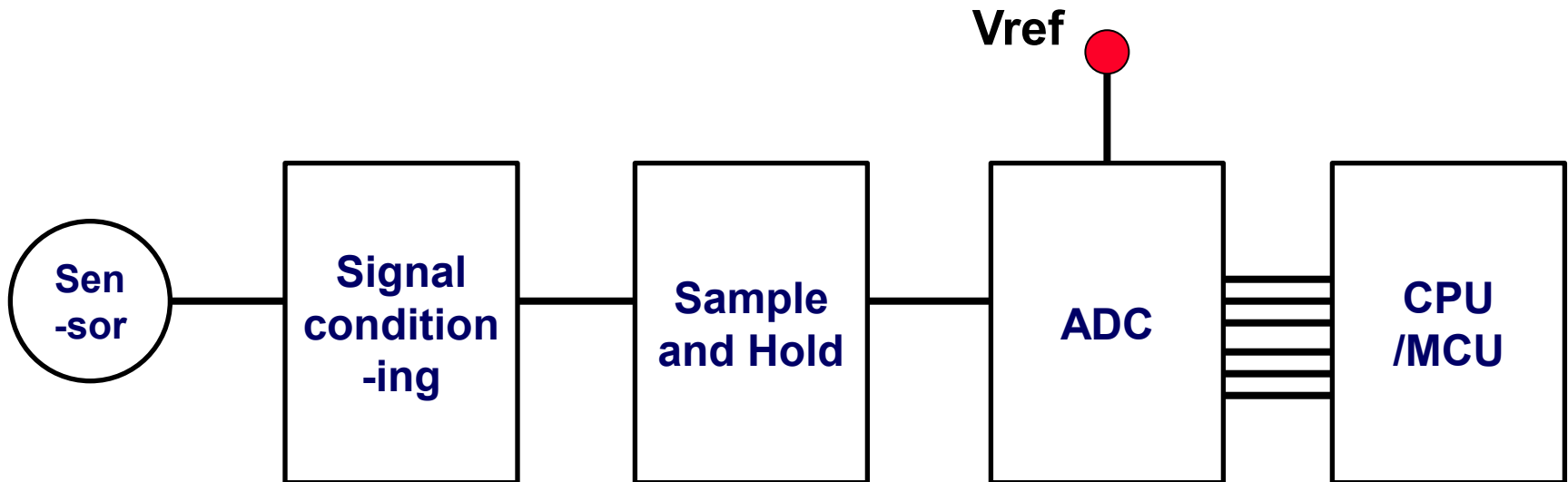
ADC/DAC

ADC dual slope integration



Interfacing with ADC

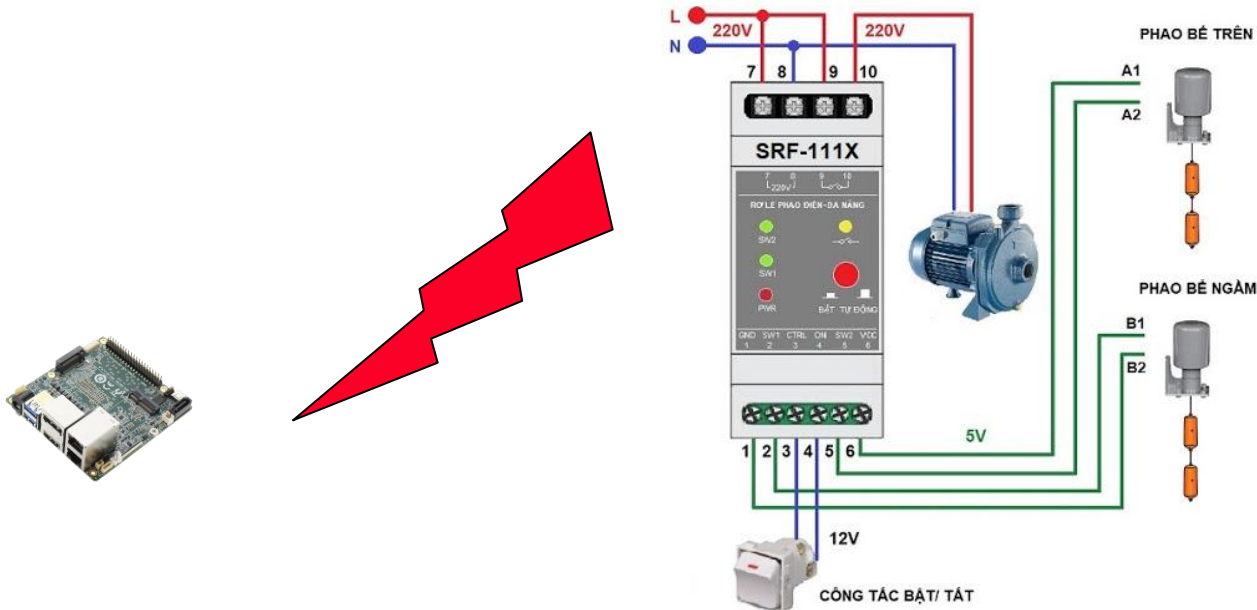
- ❑ Hardware: usually require signal conditioning, sample and hold for faster conversion.
- ❑ Software: follow the MCU specification and HAL



Standard hardware blocks for ADC interfacing

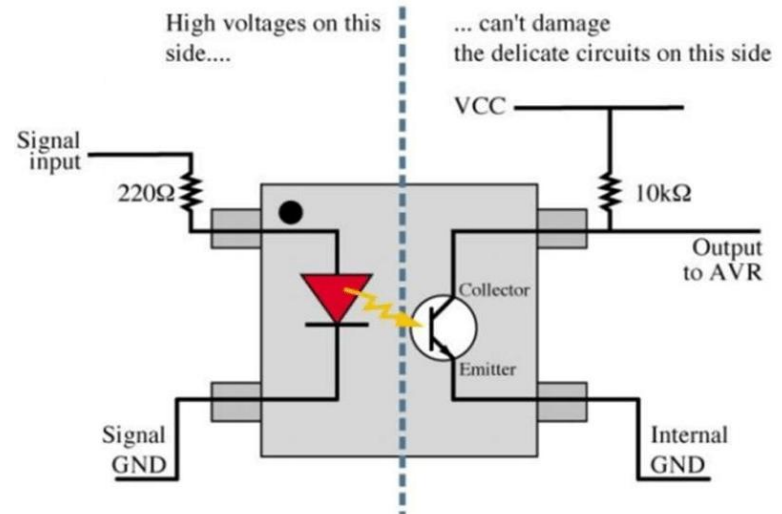
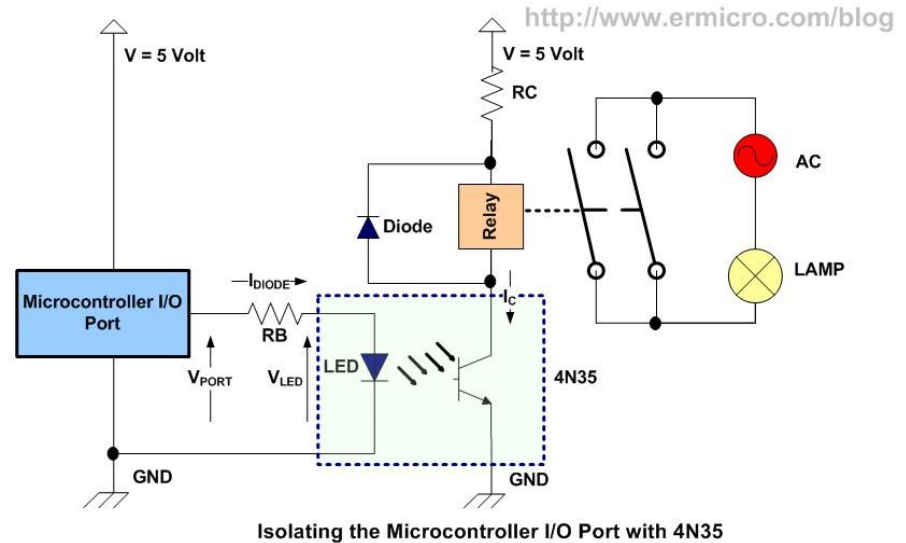
Dealing with noise and high power

- ❑ Problems when working with real-world environment
 - | High noise from motors, solenoids, magnets,...
 - | High power, high voltage cause risks to embedded system.
- ❑ Requires additional components:
 - | Noise filter: both digital and analog.
 - | Isolation circuit: relay, optocoupler.



Isolation with relay and optocoupler

- ❑ Relay: control higher power device using small current.
- ❑ Optocoupler: transfer electrical signal between two isolated circuits.



ADC on STM32F429

- ❑ STM32F429ZIT6 supports three 12-bit ADCs, each shares up to 16 channels, each channel connects to one GPIO pin.
- ❑ Total of 24 ADC pins (some are multiplexed).
- ❑ Max ADC frequency: $f_{\text{ADC}} = 30 \text{ MHz}$
- ❑ Max conversion rate:
 - | Single ADC: 2.0 MSPS (mega samples per second)
 - | Dual interleaved: 3.75 MSPS
 - | Triple interleaved: 6.0 MSPS

Basic HAL functions for ADC

HAL_ADC_Init()

HAL_ADC_DeInit()

HAL_ADC_RegisterCallback()

HAL_ADC_UnRegisterCallback()

HAL_ADC_MspInit()

HAL_ADC_MspDeInit()

HAL_ADC_ErrorCallback()

HAL_ADC_ConfigChannel()

HAL_ADC_AnalogWDGConfig()

HAL_ADC_GetState()

HAL_ADC_GetError()

HAL_ADC_Start()

HAL_ADC_Stop()

HAL_ADC_PollForConversion()

HAL_ADC_PollForEvent()

HAL_ADC_Start_IT()

HAL_ADC_Stop_IT()

HAL_ADC_IRQHandler()

HAL_ADC_Start_DMA()

HAL_ADC_Stop_DMA()

HAL_ADC_GetValue()

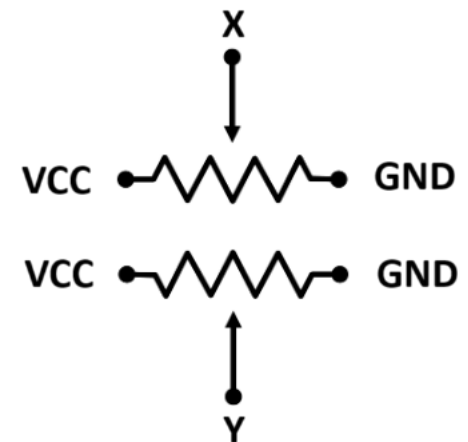
HAL_ADC_ConvCpltCallback()

HAL_ADC_ConvHalfCpltCallback()

HAL_ADC_LevelOutOfWindowCallback()

Exercise: interfacing with joystick

- ❑ Groove joystick: simple input device for 2D movement and one button.
- ❑ Hardware: two potentiometers corresponding to X, Y
- ❑ 2D movement can be measured by voltage on X and Y
 - | Requires 2 ADCs pins to read X and Y



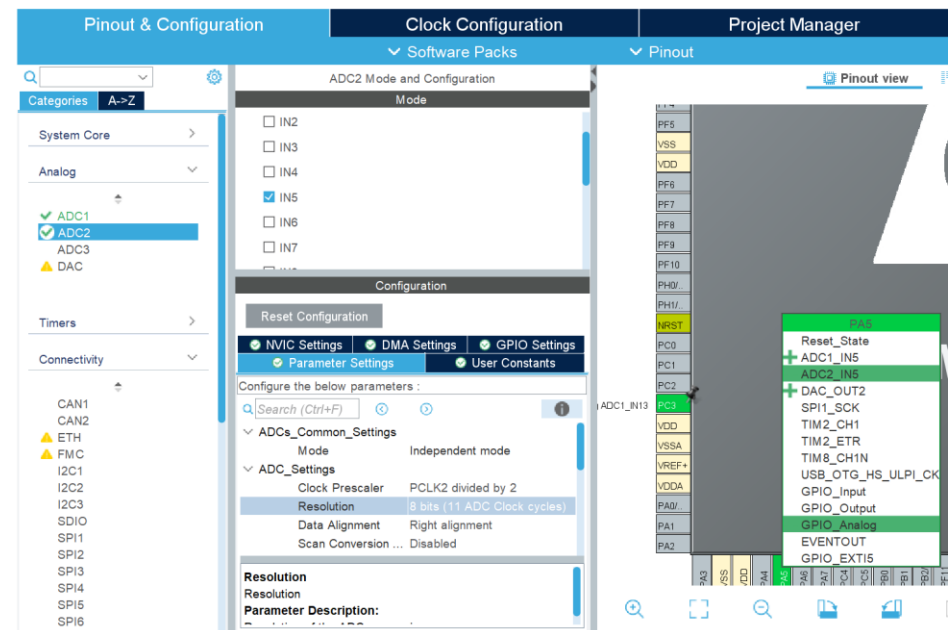
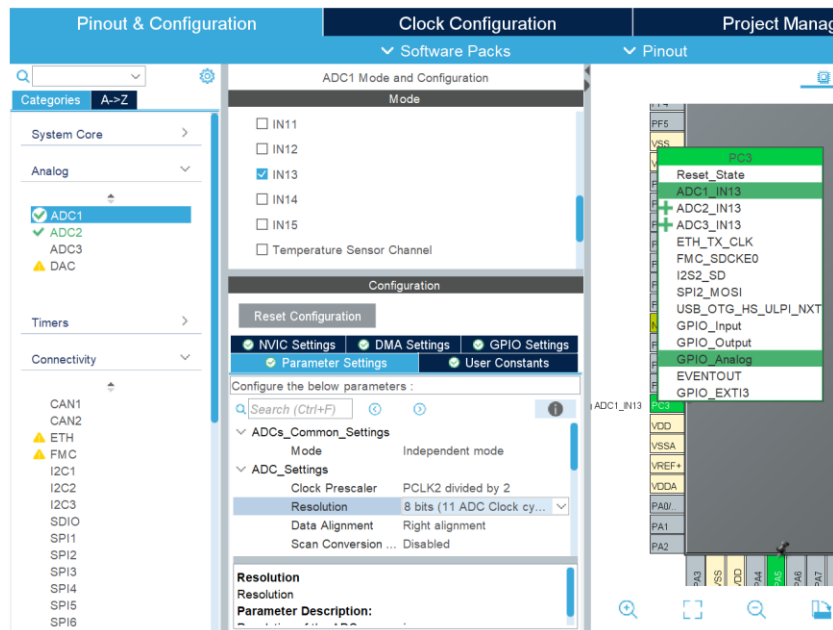
Exercise: interfacing with joystick

❑ Connection:

Joystick	STM32F429I-DISC
VCC	3V
GND	GND
X	PC3 (ADC1 IN13)
Y	PA5 (ADC2 IN5)

Exercise: interfacing with joystick

- ❑ Create new project and config IOC
 - | System clock 16-24 MHz
 - | Enable USART1 for PC communication
 - | Enable ADC1 IN13 (PC3), 8 bits mode
 - | Enable ADC2 IN5 (PA5), 8 bits mode



Exercise: interfacing with joystick

- ❑ Version 1: reading joystick values by polling ADC
- ❑ Send joystick values to PC via USART1.

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    HAL_ADC_Start(&hadc1);    //start ADC1 channel 13 (PC3)
    HAL_ADC_Start(&hadc2);    //start ADC2 channel 5 (PA5)
    HAL_ADC_PollForConversion(&hadc1, 10);
    HAL_ADC_PollForConversion(&hadc2, 10);
    int JoystickX = HAL_ADC_GetValue(&hadc1); //wait for conversion
    int JoystickY = HAL_ADC_GetValue(&hadc2); //wait for conversion
    //to-do: send this to PC or use these values to control another device
    char s[20];
    sprintf(s, "%3d-%3d\n", JoystickX, JoystickY);
    HAL_UART_Transmit(&huart1, s, strlen(s), 10);
    HAL_Delay(50);
}
/* USER CODE END WHILE */
```

Efficient Input/Output with Interrupt and DMA

- ❑ In the previous code: ADC was accessed by polling
 - | HAL_ADC_Start(), HAL_ADC_PollForConversion() are blocking functions, CPU must **wait until conversion finished to read data**.
 - | Inefficient use of CPU time.
- ❑ More efficient I/O: avoid using CPU time by utilizing interrupt-based I/O or DMA-based I/O.

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    HAL_ADC_Start(&hadc1);    //start ADC1 channel 13 (PC3)
    HAL_ADC_Start(&hadc2);    //start ADC2 channel 5 (PA5)
    HAL_ADC_PollForConversion(&hadc1, 10);
    HAL_ADC_PollForConversion(&hadc2, 10);
    int JoystickX = HAL_ADC_GetValue(&hadc1); //wait for conversion
    int JoystickY = HAL_ADC_GetValue(&hadc2); //wait for conversion
    //to-do: send this to PC or use these values to control another device
    char s[20];
    sprintf(s, "%3d-%3d\n", JoystickX, JoystickY);
    HAL_UART_Transmit(&huart1, s, strlen(s), 10);
    HAL_Delay(50);
}
/* USER CODE END WHILE */
```

Interrupt based I/O

- ❑ Hardware: ADC raises interrupt right after finishing a conversion → CPU does not need to wait.
- ❑ Software: unblocking function call to initiate ADC conversion, followed by a callback (called from interrupt service routine) to read data.
- ❑ Procedure for interrupt-based ADC reading:
 - | Enable ADC interrupt (in IOC).
 - | Implement callback function, together with data buffer and data processing logic.
 - | Trigger ADC conversion by using `HAL_ADC_Start_IT()` with pointer to callback function.
 - | When conversion is finished, interrupt will be raised and callback is called automatically.
 - | Manually trigger next ADC conversion if necessary.

Interrupt based I/O

❑ Example: ADC1 in interrupt mode

- | Enable the interrupt in ADC1's NVIC settings.
- | Declare adc1_buf và adc1_count in main.c
- | Call HAL_ADC_Start_IT() **before** main loop.

```
33⊖ /* Private define -----
34  /* USER CODE BEGIN PD */
35  #define ADC_BUF_LEN 4096
36  /* USER CODE END PD */
37
38⊖ /* Private macro -----
39  /* USER CODE BEGIN PM */
40
41  /* USER CODE END PM */
42
43  /* Private variables -----
44  ADC_HandleTypeDef hadc1;
45  ADC_HandleTypeDef hadc2;
46
47  UART_HandleTypeDef huart1;
48
49  /* USER CODE BEGIN PV */
50  uint8_t adc1_buf[ADC_BUF_LEN];
51  int adc1_count;
52  /* USER CODE END PV */
```

```
96  /* Initialize all configured peripherals */
97  MX_GPIO_Init();
98  MX_ADC1_Init();
99  MX_ADC2_Init();
100 MX_USART1_UART_Init();
101 /* USER CODE BEGIN 2 */
102 adc1_count = 0;
103 HAL_ADC_Start_IT(&hadc1);
104 /* USER CODE END 2 */
```

Interrupt based I/O

❑ Example: ADC1 in interrupt mode (cont.)

- | Enable the interrupt in ADC1's NVIC settings.
- | Declare `adc1_buf` và `adc1_count` in `main.c`
- | Call `HAL_ADC_Start_IT()` **before** main loop.
- | Implement the callback `HAL_ADC_ConvCpltCallback()`
- | Data process logic in main loop (get average from 1000 ADC reading then send to PC).

```
/* USER CODE BEGIN 4 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    if (hadc == &hadc1)
        adc1_buf[adc1_count] = HAL_ADC_GetValue(&hadc1);

    adc1_count++;
    if (adc1_count < 1000)
        HAL_ADC_Start_IT(&hadc1);
}
/* USER CODE END 4 */
```

```
/* USER CODE BEGIN 2 */
//HAL_ADC_Start_DMA(&hadc1, adc1_buf, ADC_BUF_LEN);
adc1_count = 0;
HAL_ADC_Start_IT(&hadc1);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    if (adc1_count >= 1000)
    {
        char s[20];
        int avg = 0;
        while (adc1_count > 0)
        {
            adc1_count--;
            avg += adc1_buf[adc1_count];
        }
        avg /= 1000;
        sprintf(s, "%3d\n", avg);
        HAL_UART_Transmit(&huart1, s, 4, 10);
        HAL_ADC_Start_IT(&hadc1);
    }
    HAL_Delay(10);
}
/* USER CODE END WHILE */

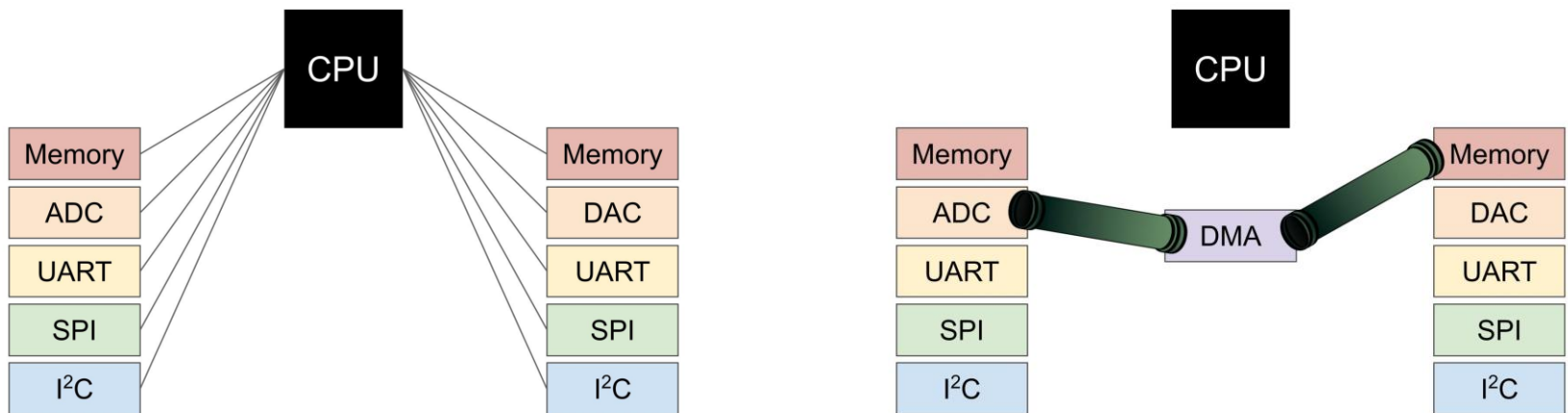
/* USER CODE BEGIN 3 */
}
```


Interrupt based I/O

- ❑ Advantage: no blocking CPU.
- ❑ Disadvantage:
 - | One interrupt per reading.
 - | HAL_ADC_GetValue() must be manually called per reading.
 - ➔ Large overhead for interrupt handling, and for large data transfer.
- ❑ A better mechanism is required for large data transfer.
 - ➔ DMA-based I/O

DMA-based I/O

- ❑ Large blocks of data are transferred directly between peripherals and memory (not passing through CPU).
- ❑ Managed by the DMA Controller (DMAC).
 - | STM32F429 has two DMACs.
 - | Eg. DMAC reads from ADC, and writes to memory simultaneously.
- ❑ Interrupt when each data block transfer is finished.
- ❑ Supports double buffer (ping pong).



[ShawnHymel](#), Maker.io

DMA-based I/O

❑ Example: DMA-based I/O for ADC1 (IN13)

- | Enable **Continuous Conversion Mode, DMA Continuous Requests**
- | Configure DMA Settings: press Add to **add DMA stream**
 - Mode Circular: ring buffer.
 - Use Fifo, Threshold Half Full: callback when buffer reaches half and full capacity.

ADC1 Mode and Configuration

Mode

☐ IN12
☒ IN13
☐ IN14

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | **DMA Settings** | GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

ADCs_Common_Settings

Mode Independent mode

ADC_Settings

Clock Prescaler PCLK2 divided by 8

Resolution 8 bits (11 ADC Clock cycles)

Data Alignment Right alignment

Scan Conversion Mode Disabled

Continuous Conversion Mode Enabled

Discontinuous Conversion Mode Disabled

DMA Continuous Requests Enabled

End Of Conversion Selection EOC flag at the end of single channel conversion

DMA Continuous Requests

DMA Continuous Requests

Parameter Description:

ADC1 Mode and Configuration

Mode

☐ IN12
☒ IN13
☐ IN14

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | **DMA Settings** | GPIO Settings

DMA Request	Stream	Direction	Priority
ADC1	DMA2 Stream 0	Peripheral To Memory	Low

Add Delete

DMA Request Settings

Peripheral Memory

Mode Circular Increment Address ☐ ☒

Use Fifo ☒ Threshold Half Full Data Width Byte Byte

Burst Size Single Single

DMA-based I/O

- ❑ Call HAL_ADC_Start_DMA() to start conversion.
- ❑ Call HAL_ADC_Stop_DMA() to stop.
- ❑ Implement 2 callback functions (exact prototype as required by HAL).
- ❑ To test: set breakpoint in each callback then run in debug mode.

```
/* USER CODE BEGIN 2 */
HAL_ADC_Start_DMA(&hadc1, adc1_buf, ADC_BUF_LEN);
/* USER CODE END 2 */

/* USER CODE BEGIN 4 */
// Called when first half of buffer is filled
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef* hadc) {
    if (hadc == &hadc1)
        HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
    else
        HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}

// Called when buffer is completely filled
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    if (hadc == &hadc1)
        HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
    else
        HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
/* USER CODE END 4 */
```

Exercise

- ❑ Read X and Y position from joystick and send to PC via USART1 with about 20 readings per second.

Exercise

- ❑ Connect the OLED SH1106 to CN2 on STM32F429 board, reuse the code in Lab 2 to control SH1106 module.
- ❑ Write code to use joystick to control a round shaped ball on SH1106 screen to run in the 2D dimension (x, y), in which x and y correspond to the values of joystick readings X and Y.